

Multithreading

Outline

- 1 Introduction
- 2 Class Thread: An Overview of the Thread Methods
- 3 Thread States: Life Cycle of a Thread
- 4 Thread Priorities and Thread Scheduling
- 5 Thread Synchronization
- 6 Producer/Consumer Relationship without Thread Synchronization
- 7 Producer/Consumer Relationship with Thread Synchronization
- 8 Producer/Consumer Relationship: The Circular Buffer
- 9 Daemon Threads
- 10 Runnable Interface
- 11 Thread Groups



1 Introduction

- Performing operations concurrently (in parallel)
 - We can walk, talk, breathe, digest, see, hear, smell... all at the same time
 - Computers can do this as well - download a file, print a file, receive email, run the clock
 - However, most programming languages do not allow concurrency
 - Usually limited to operating system "primitives" available to systems programmers
 - Java is unique - allows concurrency



1 Introduction

- Threads of execution
 - Each thread is a portion of a program that can execute concurrently with other threads (multithreading)
 - C and C++ are single-threaded
 - This gives Java powerful capabilities not found in C and C++
 - Example: downloading a video clip
 - Instead of having to download the entire clip then play it:
 - Download a portion, play that portion, download the next portion, play that portion... (streaming)
 - Ensure that it is done smoothly
 - Writing multithreaded programs is tricky
 - Imagine trying to read 3 books at once, by reading a few words at a time

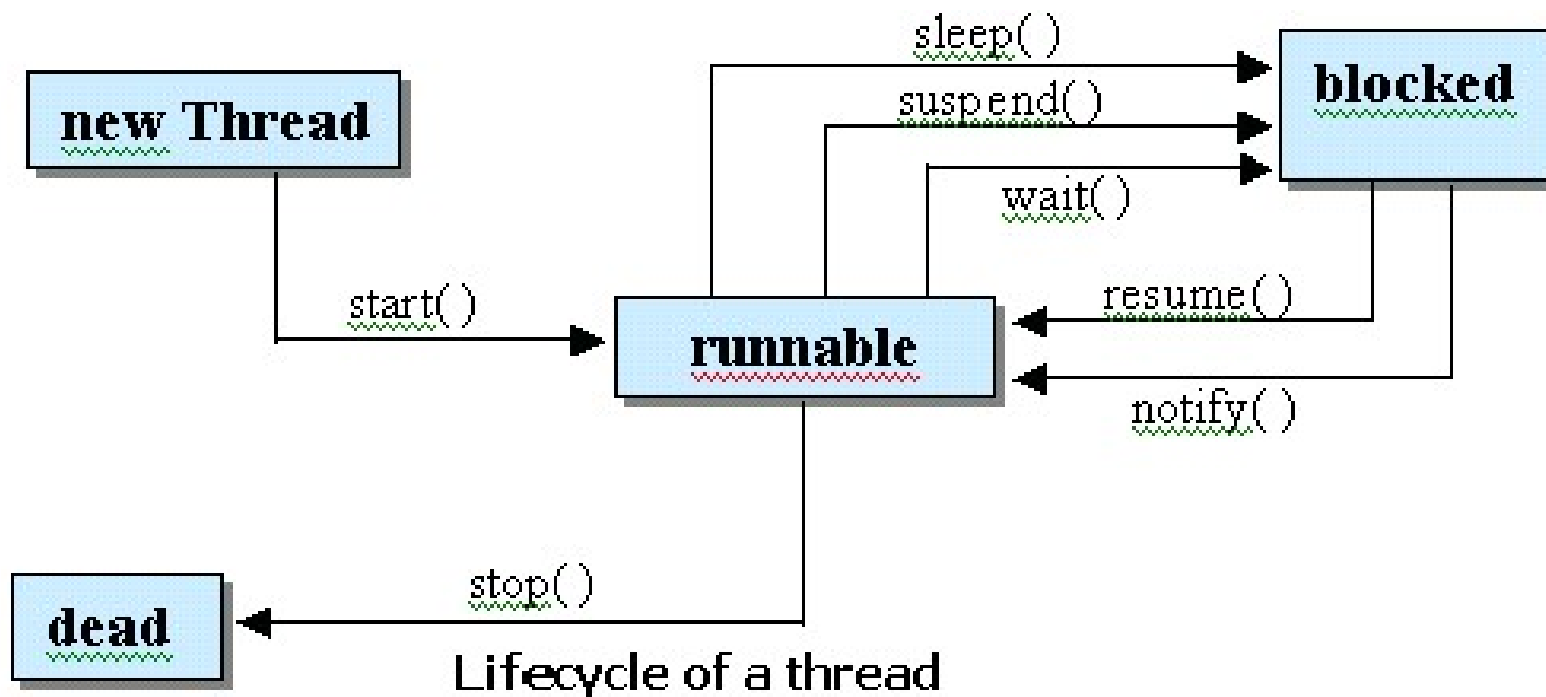


1 Introduction

- Portability
 - Differences between Solaris and Win32 (Windows95, NT, etc.)
- On Solaris
 - A thread runs to completion or until a higher priority thread becomes ready
 - Preemption occurs (processor is given to the higher-priority thread)
- On Win32
 - Threads are timesliced
 - Thread given quantum of time to execute
 - Processor then switched to any threads of equal priority
 - Preemption occurs with higher and equal priority threads



Life cycle of a Thread



2 Class Thread: An Overview of the Thread Methods

- Overview thread-related methods
 - See API for more details (especially exceptions)
 - Constructors
 - **Thread(threadName)**
 - **Thread()**
 - Creates an auto numbered **Thread** of format **Thread-1**, **Thread-2...**
 - **run**
 - "Does work" of thread
 - Can be overridden in a subclass of **Thread** or in a **Runnable** object (more on interface **Runnable** in 10)
 - **start**
 - Launches thread, then returns to caller
 - Calls **run**
 - Error to call **start** twice for same thread



Creating and Managing Threads

- The main() thread of a Java program
 - Is important because:
 - The child threads will be created from it
 - It is the last thread to finish execution. The moment the main thread stops executing, the program is terminated
 - Is automatically created with the program execution
 - Can be controlled through a thread object
- Thread can be created in two ways:
 - Declare the class as a sub-class of the class Thread, where the run() method of the Thread class needs to be overridden. [Example](#)
 - Declare a class that implements the class Runnable. Then define the run() method. [Example](#)



2 Class Thread: An Overview of the Thread Methods

- **Thread** methods
 - **static** method **sleep(milliseconds)**
 - Thread sleeps (does not contend for processor) for number of milliseconds
 - Can give lower priority threads a chance to run
 - **interrupt**
 - Interrupts a thread
 - **static** method **interrupted**
 - Returns **true** if current thread interrupted
 - **isInterrupted**
 - Determines if a thread is interrupted
 - **isAlive**
 - Returns **true** if start has been called and not dead (**run** function has not completed)



2 Class Thread: An Overview of the Thread Methods

- **Thread** methods
 - **yield** - discussed later
 - **setName(threadName)**
 - **getName**
 - **toString**
 - Returns thread name, priority, and **ThreadGroup** (more 11)
 - **static** method **currentThread**
 - Returns reference to currently executing thread
 - **join**
 - Calling thread waits for thread receiving message to die before it can proceed
 - No argument or 0 millisecond argument means thread will wait indefinitely
 - Can lead to deadlock/indefinite postponement



3 Thread States: Life Cycle of a Thread

- Thread states
 - Born state
 - Thread just created
 - When **start** called, enters ready state
 - Ready state (runnable state)
 - Highest-priority ready thread enters running state
 - Running state
 - System assigns processor to thread (thread begins executing)
 - When **run** method completes or terminates, enters dead state
 - Dead state
 - Thread marked to be removed by system
 - Entered when **run** terminates or throws uncaught exception



3 Thread States: Life Cycle of a Thread

- Other thread states
 - Blocked state
 - Entered from running state
 - Blocked thread cannot use processor, even if available
 - Common reason for blocked state - waiting on I/O request
 - Sleeping state
 - Entered when **sleep** method called
 - Cannot use processor
 - Enters ready state after sleep time expires
 - Waiting state
 - Entered when **wait** called in an object thread is accessing
 - One waiting thread becomes ready when object calls **notify**
 - **notifyAll** - all waiting threads become ready



4 Thread Priorities and Thread Scheduling

- All Java applets / applications are multithreaded
 - Threads have priority from 1 to 10
 - **Thread.MIN_PRIORITY** - 1
 - **Thread.NORM_PRIORITY** - 5 (default)
 - **Thread.MAX_PRIORITY** - 10
 - New threads inherit priority of thread that created it
- Timeslicing
 - Each thread gets a quantum of processor time to execute
 - After time is up, processor given to next thread of equal priority (if available)
 - Without timeslicing, each thread of equal priority runs to completion

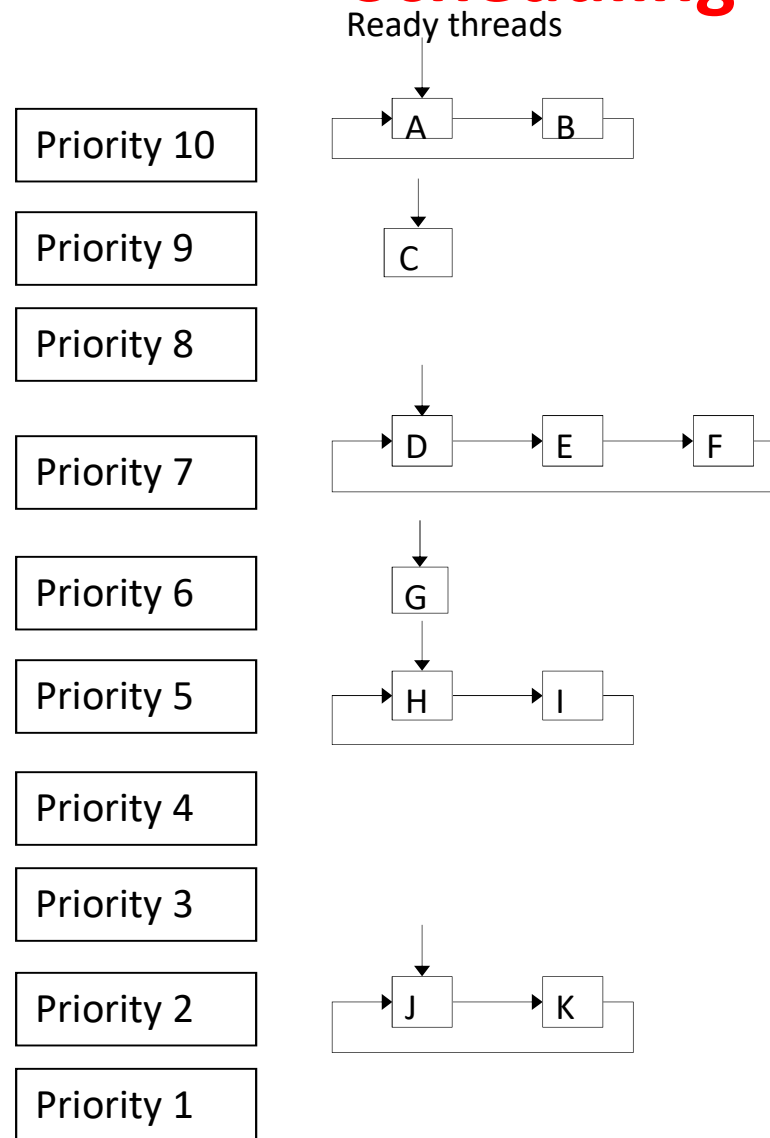


4 Thread Priorities and Thread Scheduling

- Java scheduler
 - Keeps highest-priority thread running at all times
 - If timeslicing available, ensure equal priority threads execute in round-robin fashion
 - New high priority threads could postpone execution of lower priority threads
 - Indefinite postponement (starvation)
- Priority methods
 - **setPriority(int priorityNumber)**
 - **getPriority**
 - **yield** - thread yields processor to threads of equal priority
 - Useful for non-timesliced systems, where threads run to completion



4 Thread Priorities and Thread Scheduling



4 Thread Priorities and Thread Scheduling

- Example program
 - Demonstrate basic threading techniques
 - Create a class derived from **Thread**
 - Use **sleep** method
 - Overview
 - Create four threads, which sleep for a random amount of time
 - After they finish sleeping, print their name
 - Program has two classes
 - **PrintThread**
 - Derives from **Thread**
 - Instance variable **sleepTime**
 - **ThreadTester**
 - Creates four **PrintThread** objects





Outline

Class ThreadTester

1. main

1.1 Initialize objects

1.2 start

1. extends Thread

1.1 Instance variable

```

1 // Fig. 3: ThreadTester.java
2 // Show multiple threads printing at different intervals.
3
4 public class ThreadTester {
5     public static void main( String args[] )
6     {
7         PrintThread thread1, thread2, thread3, thread4;
8
9         thread1 = new PrintThread( "thread1" );
10        thread2 = new PrintThread( "thread2" );
11        thread3 = new PrintThread( "thread3" );
12        thread4 = new PrintThread( "thread4" );
13
14        System.err.println( "\nStarting threads" );
15
16        thread1.start();
17        thread2.start();
18        thread3.start();
19        thread4.start();
20
21        System.err.println( "Threads started\n" );
22    }
23 }
24
25 class PrintThread extends Thread {
26     private int sleepTime;
27
28     // PrintThread constructor assigns name to thread
29     // by calling Thread constructor

```

main terminates after starting the **PrintThreads**, but the application does not end until the last thread dies.

Thread



Outline

1.2 Constructor

1.2.1 Randomize sleepTime

2. run

2.1 sleep

```

30 public PrintThread( String name )
31 {
32     super( name ); ← Call superclass
33                               constructor to assign
34                               name to thread.
35     // sleep between 0 and 5 seconds
36     sleepTime = (int) ( Math.random() * 5000 );
37
38     System.err.println( "Name: " + getName() +
39                               "; sleep: " + sleepTime );
40 }
41 // execute the thread
42 public void run()
43 {
44     // put thread to sleep for a random interval
45     try { ← sleep can throw an exception, so it
46         System.err.println( getName() + " going to sleep" );
47         Thread.sleep( sleepTime );
48     }
49     catch ( InterruptedException exception ) {
50         System.err.println( exception.toString() );
51     }
52
53     // print thread name
54     System.err.println( getName() + " done sleeping" );
55 }
56 }

```



Outline

Program Output

```
Name: thread1;  sleep: 1653
Name: thread2;  sleep: 2910
Name: thread3;  sleep: 4436
Name: thread4;  sleep: 201
```

```
Starting threads
Threads started
```

```
thread1 going to sleep
thread2 going to sleep
thread3 going to sleep
thread4 going to sleep
thread4 done sleeping
thread1 done sleeping
thread2 done sleeping
thread3 done sleeping
```

```
Name: thread1;  sleep: 3876
Name: thread2;  sleep: 64
Name: thread3;  sleep: 1752
Name: thread4;  sleep: 3120
```

```
Starting threads
Threads started
```

```
thread2 going to sleep
thread4 going to sleep
thread1 going to sleep
thread3 going to sleep
thread2 done sleeping
thread3 done sleeping
thread4 done sleeping
thread1 done sleeping
```

5 Thread Synchronization

- Monitors
 - An object with **synchronized** methods is a monitor
 - Any object can be a monitor
 - Methods declared **synchronized**
 - **public synchronized int myMethod(int x)**
 - Object allows only one thread to execute a **synchronized method** at a time
 - Obtaining the lock and locking an object
 - If multiple **synchronized** methods, only one may be active
 - Java also has **synchronized** blocks of code (more 10)



5 Thread Synchronization

- Thread may decide it cannot proceed
 - May voluntarily call **wait** while accessing a **synchronized** method
 - Removes thread from contention for monitor object and processor
 - Thread in waiting state
 - Other threads try to enter monitor object
 - Suppose condition first thread needs has now been met
 - Can call **notify** to tell a waiting thread to enter ready state
 - **notifyAll** - tells all waiting threads to enter ready state



6 Producer/Consumer Relationship without Thread Synchronization

- Producer / Consumer relationship
 - Producing thread may write to buffer (shared memory)
 - Consuming thread reads from buffer
 - If not synchronized, data can become corrupted
 - Producer may write before consumer read last data
 - Data lost
 - Consumer may read before producer writes new data
 - Data "doubled"
 - Using synchronization
 - If producer knows that consumer has not read last data, calls **wait** (awaits a **notify** command from consumer)
 - If consumer knows producer has not updated data, calls **wait** (awaits **notify** command from producer)



6 Producer/Consumer Relationship without Thread Synchronization

- Example
 - Producer / Consumer relationship without synchronization
 - Overview
 - Producer writes the numbers 1 through 10 to a buffer
 - Consumer reads them from buffer and sums them
 - If producer/consumer operate in order, total should be 55
 - Classes
 - **ProduceInteger** and **ConsumeInteger**
 - Inherit from **Thread**
 - **sleep** for random amount of time, then read from / write to buffer
 - **HoldIntegerUnsyncronized**
 - Has data and unsynchronized set and get methods
 - **SharedCell**
 - Driver, creates threads and calls **start**





Outline



Class SharedCell

1. main

1.1 Initialize objects

```
1 // Fig. 4: SharedCell.java
2 // Show multiple threads modifying shared object.
3 public class SharedCell {
4     public static void main( String args[] )
5     {
6         HoldIntegerUnsynchronized h =
7             new HoldIntegerUnsynchronized();
8         ProduceInteger p = new ProduceInteger( h );
9         ConsumeInteger c = new ConsumeInteger( h );
10
11         p.start();
12         c.start();
13     }
14 }
15
```



Outline

Class ProduceInteger

1. extends Thread

variable

2. run

2.1 Randomize sleep

2.2 setSharedInt

```

16 // Fig. 4: ProduceInteger.java
17 // Definition of threaded class ProduceInteger
18 public class ProduceInteger extends Thread {
19     private HoldIntegerUnsynchronized pHold;
20
21     public ProduceInteger( HoldIntegerUnsynchronized h )
22     {
23         super( "ProduceInteger" );
24         pHold = h;
25     }
26
27     public void run()
28     {
29         for ( int count = 1; count <= 10; count++ ) {
30             // sleep for a random interval
31             try {
32                 Thread.sleep( (int) ( Math.random() * 3000 ) );
33             }
34             catch( InterruptedException e ) {
35                 System.err.println( e.toString() );
36             }
37
38             pHold.setSharedInt( count );
39         }
40
41         System.err.println( getName() +
42             " finished producing values" +
43             "\nTerminating " + getName() );
44     }
45 }
46

```

pHold refers to a **HoldIntegerUnsynchronized** object, and will use its set methods.



Outline

Class ConsumeInteger

1. extends Thread

1.1 Instance variable

2. run

2.1 Randomize sleep

2.2 getSharedInt

```
47 // Fig. 4: ConsumeInteger.java
48 // Definition of threaded class ConsumeInteger
49 public class ConsumeInteger extends Thread {
50     private HoldIntegerUnsynchronized cHold;
51
52     public ConsumeInteger( HoldIntegerUnsynchronized h )
53     {
54         super( "ConsumeInteger" );
55         cHold = h;
56     }
57
58     public void run()
59     {
60         int val, sum = 0;
61
62         do {
63             // sleep for a random interval
64             try {
65                 Thread.sleep( (int) ( Math.random() * 3000 ) );
66             }
67             catch( InterruptedException e ) {
68                 System.err.println( e.toString() );
69             }
70
71             val = cHold.getSharedInt();
72             sum += val;
73         } while ( val != 10 );
74
75         System.err.println(
76             getName() + " retrieved values totaling: " + sum +
77             "\nTerminating " + getName() );
78     }
79 }
```



Outline

Class HoldInteger Unsynchronized

1. Instance variable

2. setSharedInt (unsynchronized)

3. getSharedInt (unsynchronized)

```
1 // Fig. 4: HoldIntegerUnsynchronized.java
2 // Definition of class HoldIntegerUnsynchronized
3 public class HoldIntegerUnsynchronized {
4     private int sharedInt = -1;
5
6     public void setSharedInt( int val )
7     {
8         System.err.println( Thread.currentThread().getName() +
9             " setting sharedInt to " + val );
10        sharedInt = val;
11    }
12
13    public int getSharedInt()
14    {
15        System.err.println( Thread.currentThread().getName() +
16            " retrieving sharedInt value " + sharedInt );
17        return sharedInt;
18    }
19 }
```

Because the set and get methods are unsynchronized, the two threads could call them at the same time.



Outline

Program Output

```
ConsumeInteger retrieving sharedInt value -1
ConsumeInteger retrieving sharedInt value -1
ProduceInteger setting sharedInt to 1
ProduceInteger setting sharedInt to 2
ConsumeInteger retrieving sharedInt value 2
ProduceInteger setting sharedInt to 3
ProduceInteger setting sharedInt to 4
ProduceInteger setting sharedInt to 5
ConsumeInteger retrieving sharedInt value 5
ProduceInteger setting sharedInt to 6
ProduceInteger setting sharedInt to 7
ProduceInteger setting sharedInt to 8
ConsumeInteger retrieving sharedInt value 8
ConsumeInteger retrieving sharedInt value 8
ProduceInteger setting sharedInt to 9
ConsumeInteger retrieving sharedInt value 9
ConsumeInteger retrieving sharedInt value 9
ProduceInteger setting sharedInt to 10
ProduceInteger finished producing values
Terminating ProduceInteger
ConsumeInteger retrieving sharedInt value 10
ConsumeInteger retrieved values totaling: 49
Terminating ConsumeInteger
```

Notice how the producer and consumer act out of order, which results in a sum of 49 (not 55).

7 Producer/Consumer Relationship with Thread Synchronization

- Condition variable of a monitor
 - Variable used to test some condition
 - Determines if thread should call **wait**
 - For our producer / consumer relationship
 - Condition variable determines whether the producer should write to buffer or if consumer should read from buffer
 - Use boolean variable **writable**
 - If **writable true**, producer can write to buffer
 - If **false**, then producer calls **wait**, and awaits **notify**
 - If **writable false**, consumer can read from buffer
 - If **true**, consumer calls **wait**



7 Producer/Consumer Relationship with Thread Synchronization

- Redo example program with synchronization
 - Synchronize the set and get methods
 - Once the producer writes to memory, **writable** is **false** (cannot write again)
 - Once consumer reads, **writable** is **true** (cannot read again)
 - Each thread relies on the other to toggle **writable** and call **notify**
 - Only Class **HoldIntegerUnsynchronized** is changed
 - Now called **HoldIntegerSynchronized**
 - We only changed the implementation of the set and get methods



Classes SharedCell,
ConsumeInteger, and
ProduceInteger same
as before

Class HoldInteger
Synchronized

1. Instance variables

2. setSharedInt
(synchronized)

```

1 // Fig. 5: HoldIntegerSynchronized.java
2 // Definition of class HoldIntegerSynchronized that
3 // uses thread synchronization to ensure that both
4 // threads access sharedInt at the proper times.
5 public class HoldIntegerSynchronized {
6     private int sharedInt = -1;
7     private boolean
8
9     public synchronized
10    {
11        while ( !writeable ) { // not the producer's turn
12            try {
13                wait();
14            }
15            catch ( InterruptedException e ) {
16                e.printStackTrace();
17            }
18        }
19
20        System.err.println( Thread.currentThread().getName() +
21            " setting sharedInt to " + val );
22        sharedInt = val;
23
24        writeable = false;
25        notify(); // tell a waiting thread to k
26    }
27

```

Test the condition variable. If it is not the producer's turn, then **wait**.

If **writeable** is **true**, write to the buffer, toggle **writeable**, and **notify** any waiting threads (so they may read from the buffer).

3. getSharedInt (synchronized)

```
28 public synchronized int getSharedInt()
29 {
30     while ( writeable ) {    // not the consumer's turn
31         try {
32             wait();
33         }
34         catch ( InterruptedException e ) {
35             e.printStackTrace();
36         }
37     }
38
39     writeable = true;
40     notify(); // tell a waiting thread to become ready
41
42     System.err.println( Thread.currentThread().getName() +
43         " retrieving sharedInt value " + sharedInt );
44     return sharedInt;
45 }
46 }
```

As with **setSharedInt**, test the condition variable. If not the consumer's turn, then **wait**.

If it is ok to read (**writeable** is **false**), set **writeable** to **true**, **notify**, and **return sharedInt**.



Outline

Program Output

```
ProduceInteger setting sharedInt to 1
ConsumeInteger retrieving sharedInt value 1
ProduceInteger setting sharedInt to 2
ConsumeInteger retrieving sharedInt value 2
ProduceInteger setting sharedInt to 3
ConsumeInteger retrieving sharedInt value 3
ProduceInteger setting sharedInt to 4
ConsumeInteger retrieving sharedInt value 4
ProduceInteger setting sharedInt to 5
ConsumeInteger retrieving sharedInt value 5
ProduceInteger setting sharedInt to 6
ConsumeInteger retrieving sharedInt value 6
ProduceInteger setting sharedInt to 7
ConsumeInteger retrieving sharedInt value 7
ProduceInteger setting sharedInt to 8
ConsumeInteger retrieving sharedInt value 8
ProduceInteger setting sharedInt to 9
ConsumeInteger retrieving sharedInt value 9
ProduceInteger setting sharedInt to 10
ProduceInteger finished producing values
Terminating ProduceInteger
ConsumeInteger retrieving sharedInt value 10
ConsumeInteger retrieved values totaling: 55
Terminating ConsumeInteger
```

The producer and consumer act in order, and the proper total is reached (55).

8 Producer/Consumer Relationship: The Circular Buffer

- Previous program
 - Does access data properly, but not optimally
 - Producer cannot produce faster than consumer can consume
 - To allow this, use a circular buffer
 - Has enough cells to handle "extra" production
 - Once producer knows consumer has read data, allowed to overwrite it
- Redo program with a circular buffer
 - For the circular buffer, use a 5-element array
 - Have variables **readLoc** and **writeLoc** to keep track of where in array producer and consumer are
 - Incremented, and kept between 0 and 4 with % 5
 - Condition variables **readable** and **writable**



8 Producer/Consumer Relationship: The Circular Buffer

- Redo program with a circular buffer
 - Producer starts first, so **writeLoc** > **readLoc** (in beginning)
 - If **writeLoc** == **readLoc** (in set method), producer looped around and "caught up" to consumer
 - Buffer is full, so producer stops writing (**wait**)
 - In get method
 - If **readLoc** == **writeLoc** then consumer "caught up" to producer
 - Buffer is empty, so consumer stops reading (**wait**)
 - This time, use a GUI
 - Only the set and get methods (in **HoldIntegerSynchronized**) change significantly





Outline

Class SharedCell

1. GUI added

```
1 // Fig. 6: SharedCell.java
2 // Show multiple threads modifying shared object.
3 import java.text.DecimalFormat;
4 import java.awt.*;
5 import java.awt.event.*;
6 import javax.swing.*;
7
8 public class SharedCell extends JFrame {
9     public SharedCell()
10    {
11        super( "Demonstrating Thread Synchronization" );
12        JTextArea output = new JTextArea( 20, 30 );
13
14        getContentPane().add( new JScrollPane( output ) );
15        setSize( 500, 500 );
16        show();
17
18        // set up threads and start threads
19        HoldIntegerSynchronized h =
20            new HoldIntegerSynchronized( output );
21        ProduceInteger p = new ProduceInteger( h, output );
22        ConsumeInteger c = new ConsumeInteger( h, output );
23
24        p.start();
25        c.start();
26    }
27
```



Outline

1. GUI added

```
28 public static void main( String args[] )
29 {
30     SharedCell app = new SharedCell();
31     app.addWindowListener(
32         new WindowAdapter() {
33             public void windowClosing( WindowEvent e )
34             {
35                 System.exit( 0 );
36             }
37         }
38     );
39 }
40 }
41
```



Outline

Class ProduceInteger

1. Instance variable added to accommodate GUI

```
42 // Fig. 6: ProduceInteger.java
43 // Definition of threaded class ProduceInteger
44 import javax.swing.JTextArea;
45
46 public class ProduceInteger extends Thread {
47     private HoldIntegerSynchronized pHold;
48     private JTextArea output;
49
50
51     public ProduceInteger( HoldIntegerSynchronized h,
52                           JTextArea o )
53     {
54         super( "ProduceInteger" );
55         pHold = h;
56         output = o;
57     }
58
59     public void run()
60     {
61         for ( int count = 1; count <= 10; count++ ) {
62             // sleep for a random interval
63             // Note: Interval shortened purposely to fill buffer
64             try {
65                 Thread.sleep( (int) ( Math.random() * 500 ) );
66             }
67             catch( InterruptedException e ) {
68                 System.err.println( e.toString() );
69             }
70
71             pHold.setSharedInt( count );
72         }
```



Outline

1.1 Update GUI

```
73
74     output.append( "\n" + getName() +
75         " finished producing values" +
76         "\nTerminating " + getName() + "\n" );
77 }
78 }
79
```



Outline

Class ConsumeInteger

1. Instance variable added to accomodate GUI

```
80 // Fig. 6: ConsumeInteger.java
81 // Definition of threaded class ConsumeInteger
82 import javax.swing.JTextArea;
83
84 public class ConsumeInteger extends Thread {
85     private HoldIntegerSynchronized cHold;
86     private JTextArea output;
87
88     public ConsumeInteger( HoldIntegerSynchronized h,
89                           JTextArea o )
90     {
91         super( "ConsumeInteger" );
92         cHold = h;
93         output = o;
94     }
95
96     public void run()
97     {
98         int val, sum = 0;
99
100         do {
101             // sleep for a random interval
102             try {
103                 Thread.sleep( (int) ( Math.random() * 3000 ) );
104             }
105             catch( InterruptedException e ) {
106                 System.err.println( e.toString() );
107             }
108
109             val = cHold.getSharedInt();
110             sum += val;
111         } while ( val != 10 );
```



Outline

1.1 Update GUI

```
112
113     output.append( "\n" + getName() +
114         " retrieved values totaling: " + sum +
115         "\nTerminating " + getName() + "\n" );
116 }
117 }
118
```




Outline

Class HoldInteger Synchronized

1. Instance variables

```

119// Fig. 6: HoldIntegerSynchronized.java
120// Definition of class HoldIntegerSynchronized that
121// uses thread synchronization to ensure that both
122// threads access sharedInt at the proper times.
123import javax.swing.JTextArea;
124import java.text.DecimalFormat;
125
126public class HoldIntegerSynchronized {
127    private int sharedInt[] = { -1, -1, -1, -1, -1 };
128    private boolean writeable = true;
129    private boolean readable = false;
130    private int readLoc = 0, writeLoc = 0;
131    private JTextArea output;
132
133    public HoldIntegerSynchronized( JTextArea
134    {
135        output = o;
136    }
137
138    public synchronized void setSharedInt( int val )
139    {
140        while ( !writeable ) {
141            try {
142                output.append( " WAITING TO PRODUCE " + val );
143                wait();
144            }
145            catch ( InterruptedException e ) {
146                System.err.println( e.toString() );
147            }
148        }
149

```

Notice all the added instance variables, including the circular buffer and condition variables.

Int



```

150     sharedInt[ writeLoc ] = val;
151     readable = true;
152
153     output.append( "\nProduced " + val +
154                   " into cell " + writeLoc );
155
156     writeLoc = ( writeLoc + 1 ) % 5;
157
158     output.append( "\twrite " + writeLoc +
159                   "\tread " + readLoc);
160     displayBuffer( output, sharedInt );
161
162     if ( writeLoc == readLoc ) {
163         writeable = false;
164         output.append( "\nBUFFER FULL" );
165     }
166
167     notify();
168 }
169
170 public synchronized int getSharedInt()
171 {
172     int val;
173
174     while ( !readable ) {
175         try {
176             output.append( " WAITING TO CONSUME" );
177             wait();
178         }
179         catch ( InterruptedException e ) {
180             System.err.println( e.toString() );
181         }
182     }

```

Set appropriate location in the circular buffer. Update **readable**.

Increment **writeLoc**, use % 5 to keep it in range.

Test for full buffer.



Outline

3. getSharedInt

4. GUI method

```

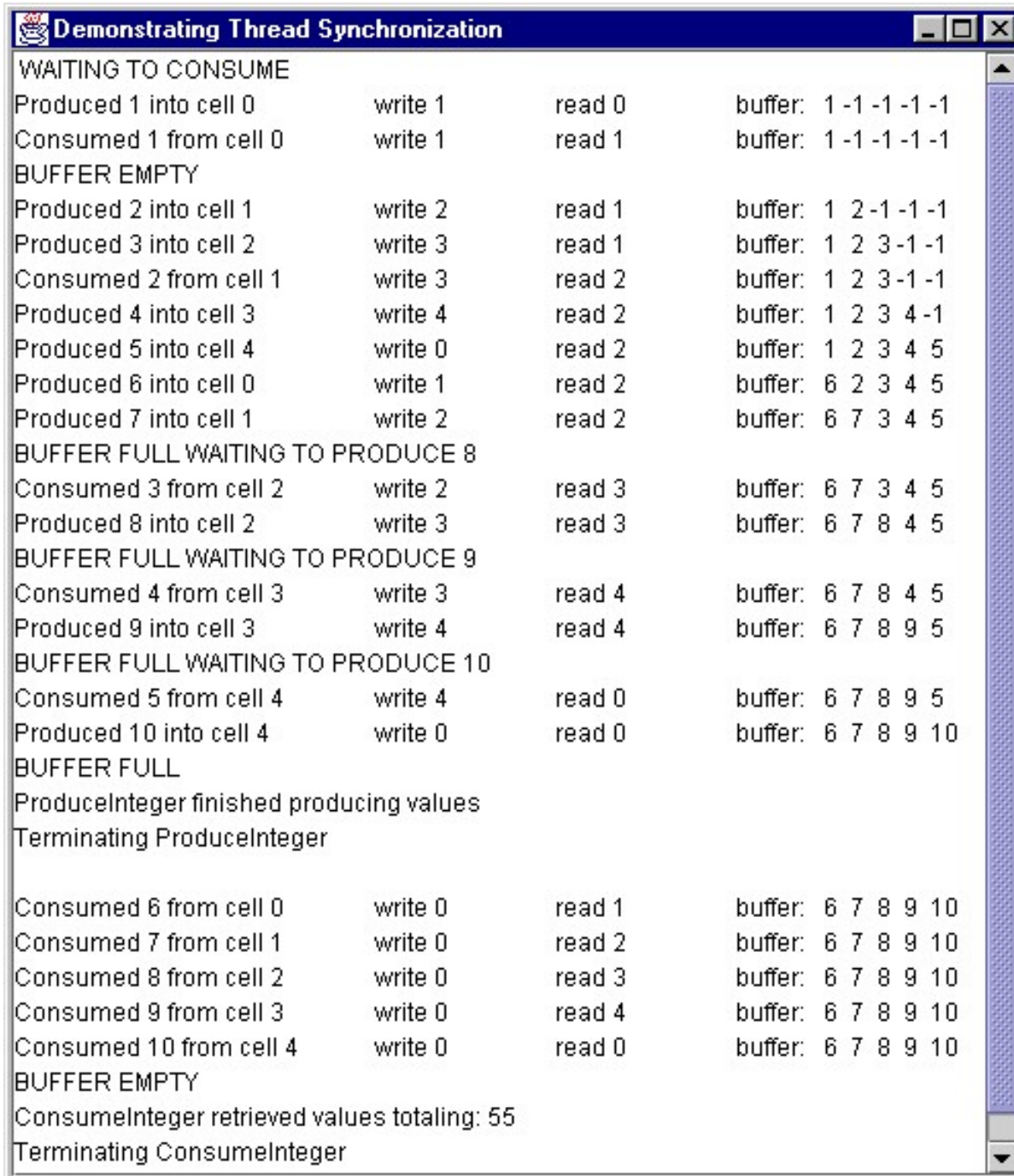
183
184     writeable = true;
185     val = sharedInt[ readLoc ];
186
187     output.append( "\nConsumed " + val +
188                   " from cell " + readLoc );
189
190     readLoc = ( readLoc + 1 ) % 5;
191
192     output.append( "\twrite " + writeLoc +
193                   "\tread " + readLoc );
194     displayBuffer( output, sharedInt );
195
196     if ( readLoc == writeLoc ) {
197         readable = false;
198         output.append( "\nBUFFER EMPTY" );
199     }
200
201     notify();
202
203     return val;
204 }
205
206 public void displayBuffer( JTextArea out, int buf[] )
207 {
208     DecimalFormat formatNumber = new DecimalFormat( " #;-#" );
209     output.append( "\tbuffer: " );
210
211     for ( int i = 0; i < buf.length; i++ )
212         out.append( " " + formatNumber.format( buf[ i ] ) );
213 }
214 }

```

Similar to **setSharedInt**. Update **readLoc**, test for empty buffer, **return val**.

Outline

Program Output



```

Demonstrating Thread Synchronization
WAITING TO CONSUME
Produced 1 into cell 0      write 1      read 0      buffer: 1 -1 -1 -1 -1
Consumed 1 from cell 0     write 1      read 1      buffer: 1 -1 -1 -1 -1
BUFFER EMPTY
Produced 2 into cell 1     write 2      read 1      buffer: 1 2 -1 -1 -1
Produced 3 into cell 2     write 3      read 1      buffer: 1 2 3 -1 -1
Consumed 2 from cell 1     write 3      read 2      buffer: 1 2 3 -1 -1
Produced 4 into cell 3     write 4      read 2      buffer: 1 2 3 4 -1
Produced 5 into cell 4     write 0      read 2      buffer: 1 2 3 4 5
Produced 6 into cell 0     write 1      read 2      buffer: 6 2 3 4 5
Produced 7 into cell 1     write 2      read 2      buffer: 6 7 3 4 5
BUFFER FULL WAITING TO PRODUCE 8
Consumed 3 from cell 2     write 2      read 3      buffer: 6 7 3 4 5
Produced 8 into cell 2     write 3      read 3      buffer: 6 7 8 4 5
BUFFER FULL WAITING TO PRODUCE 9
Consumed 4 from cell 3     write 3      read 4      buffer: 6 7 8 4 5
Produced 9 into cell 3     write 4      read 4      buffer: 6 7 8 9 5
BUFFER FULL WAITING TO PRODUCE 10
Consumed 5 from cell 4     write 4      read 0      buffer: 6 7 8 9 5
Produced 10 into cell 4    write 0      read 0      buffer: 6 7 8 9 10
BUFFER FULL
ProduceInteger finished producing values
Terminating ProduceInteger

Consumed 6 from cell 0     write 0      read 1      buffer: 6 7 8 9 10
Consumed 7 from cell 1     write 0      read 2      buffer: 6 7 8 9 10
Consumed 8 from cell 2     write 0      read 3      buffer: 6 7 8 9 10
Consumed 9 from cell 3     write 0      read 4      buffer: 6 7 8 9 10
Consumed 10 from cell 4    write 0      read 0      buffer: 6 7 8 9 10
BUFFER EMPTY
ConsumerInteger retrieved values totaling: 55
Terminating ConsumerInteger
  
```

9 Daemon Threads

- Daemon threads
 - Thread that runs for benefit of other threads
 - Garbage collector
 - Run in background
 - Use processor time that would otherwise go to waste
 - Unlike normal threads, do not prevent a program from terminating
 - When only daemon threads remain, program exits
 - Must designate a thread as daemon before **start** called
`setDaemon( true );`
 - Method **isDaemon**
 - Returns **true** if thread is a daemon thread



10 Runnable Interface

- Java does not support multiple inheritance
 - Instead, use interfaces (Chapter 9)
 - Until now, inherited from class **Thread**, overrode **run**
- Multithreading for an already derived class
 - Implement interface **Runnable** (**java.lang**)
 - New class objects "are" **Runnable** objects
 - Override **run** method
 - Controls thread, just as deriving from **Thread** class
 - In fact, class **Thread** implements interface **Runnable**
 - Create new threads using **Thread** constructors
 - **Thread(runnableObject)**
 - **Thread(runnableObject, threadName)**



10 Runnable Interface

- Synchronized blocks of code

```
synchronized( monitorObject ){  
    ...  
}
```

- **monitorObject**- Object to be locked while thread executes block of code

- Suspending threads

- In earlier versions of Java, there were methods to suspend/resume threads
 - Dangerous, can lead to deadlock
- Instead, use **wait** and **notify**



10 Runnable Interface

- Upcoming example program
 - Create a GUI and three threads, each constantly displaying a random letter
 - Have suspend buttons, which will suspend a thread
 - Actually calls **wait**
 - When suspend unclicked, calls **notify**
 - Use an array of **booleans** to keep track of which threads are suspended





Outline

Class RandomCharacters

1. implements Runnable

Use a **boolean** array to keep track of which threads are "suspended". We will actually use **wait** and **notify** to suspend the threads.

1.2 init

```

1 // Fig. 7: RandomCharacters.java
2 // Demonstrating the Runnable interface
3 import java.awt.*;
4 import java.awt.event.*;
5 import javax.swing.*;
6
7 public class RandomCharacters extends JApplet
8         implements Runnable,
9         ActionListener {
10     private String alphabet = "ABCDEFGHIJ";
11     private JLabel outputs[];
12     private JCheckBox checkboxes[];
13     private final static int SIZE = 3;
14
15     private Thread threads[];
16     private boolean suspended[];
17
18     public void init()
19     {
20         outputs = new JLabel[ SIZE ];
21         checkboxes = new JCheckBox[ SIZE ];
22
23         threads = new Thread[ SIZE ];
24         suspended = new boolean[ SIZE ];
25
26         Container c = getContentPane();
27         c.setLayout( new GridLayout( SIZE, 2, 5, 5 ) );
28

```



```

29     for ( int i = 0; i < SIZE; i++ ) {
30         outputs[ i ] = new JLabel();
31         outputs[ i ].setBackground( Color.green );
32         outputs[ i ].setOpaque( true );
33         c.add( outputs[ i ] );
34
35         checkboxes[ i ] = new JCheckBox( "Sus"
36         checkboxes[ i ].addActionListener( th
37         c.add( checkboxes[ i ] );
38     }
39 }
40
41 public void start()
42 {
43     // create threads and start every time start is called
44     for ( int i = 0; i < threads.length; i++ ) {
45         threads[ i ] =
46             new Thread( this, "Thread " + i );
47         threads[ i ].start();
48     }
49 }
50
51 public void run()
52 {
53     Thread currentThread = Thread.currentThread();
54     int index = getIndex( currentThread );
55     char displayChar;
56
57     while ( threads[ index ] == currentThread ) {
58         // sleep from 0 to 1 second
59         try {
60             Thread.sleep( (int) ( Math.random() * 1000 ) );

```

Use the **Thread** constructor to create new threads. The **Runnable** object is **this**, because the applet implements interface **Runnable**.

In the **start** method of the applet call the **start** method of the threads.

2.2 start

Loop will execute indefinitely because **threads[index] == currentThread**. The **stop** method in the applet sets all threads to **null**, which causes the loop to end.

3.1 synchronized block

Synchronized block tests suspended array to see if a thread should be "suspended". If so, calls **wait**.

andom

4. getIndex

```

62         synchronized( this ) {
63             while ( suspended[ index ] &&
64                 threads[ index ] == currentThread )
65                 wait();
66         }
67     }
68     catch ( InterruptedException e ) {
69         System.err.println( "sleep interrupted" );
70     }
71
72     displayChar = alphabet.charAt(
73         (int) ( Math.random() * 26 ) );
74
75     outputs[ index ].setText( currentThread.getName() +
76         ": " + displayChar );
77 }
78
79 System.err.println(
80     currentThread.getName() + " terminating" );
81 }
82
83 private int getIndex( Thread current )
84 {
85     for ( int i = 0; i < threads.length; i++ )
86         if ( current == threads[ i ] )
87             return i;
88
89     return -1;
90 }
91

```

5. stop

```

92  public synchronized void stop()
93  {
94      // stop threads every time stop is called
95      // as the user browses another Web page
96      for ( int i = 0; i < threads.length; i++ )
97          threads[ i ] = null;
98
99      notifyAll();
100 }
101
102 public synchronized void actionPerformed((ActionEvent e)
103 {
104     for ( int i = 0; i < checkboxes.length; i++ ) {
105         if ( e.getSource() == checkboxes[ i ] ) {
106             suspended[ i ] = !suspended[ i ];
107
108             outputs[ i ].setBackground(
109                 !suspended[ i ] ? Color.green : Color.red );
110
111             if ( !suspended[ i ] )
112                 notify();
113
114             return;
115         }
116     }
117 }
118 }

```

Sets all threads to **null**, which causes loop in **run** to end, and **run** terminates.

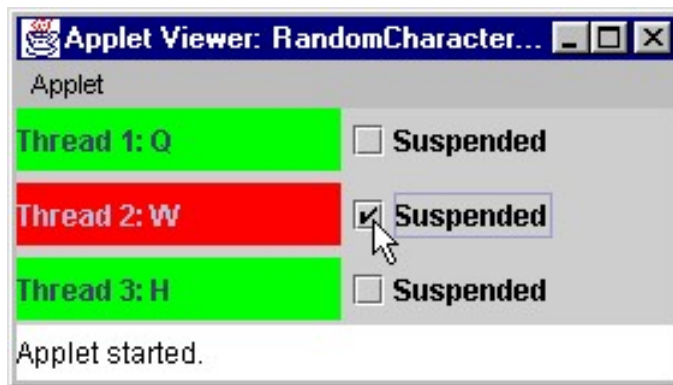
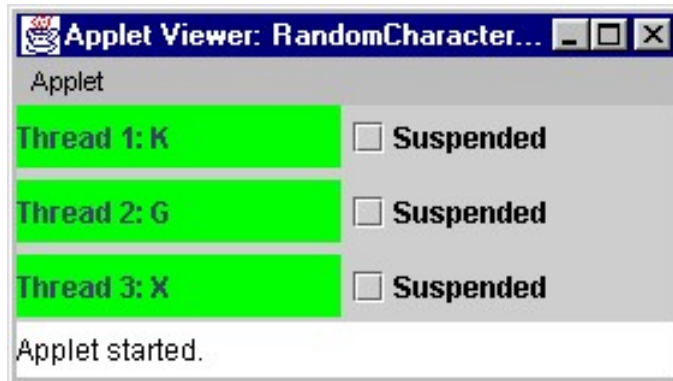
Loop and find which box was checked, and suspend appropriate thread. The **run** method checks for suspended threads.

If suspend is off, then notify the appropriate thread.



Outline

Program Output



11 Thread Groups

- Thread groups
 - Threads in a thread group can be dealt with as a group
 - May want to **interrupt** all threads in a group
 - Thread group can be parent to a child thread group
- Class **ThreadGroup**
 - Constructors
 - ThreadGroup(threadGroupName)**
 - ThreadGroup(parentThreadGroup, name)**
 - Creates child **ThreadGroup** named **name**



11 Thread Groups

- Associating **Threads** with **ThreadGroups**
 - Use constructors
 - **Thread(threadGroup, threadName)**
 - **Thread(threadGroup, runnableObject)**
 - Invokes **run** method of **runnableObject** when thread executes
 - **Thread(threadGroup, runnableObject, threadName)**
 - As above, but **Thread** named **threadName**



11 Thread Groups

- **ThreadGroup** Methods
 - See API for more details
 - **activeCount**
 - Number of active threads in a group and all child groups
 - **enumerate**
 - Two versions copy active threads into an array of references
 - Two versions copy active threads in a child group into an array of references
 - **getMaxPriority**
 - Returns maximum priority of a **ThreadGroup**
 - **setMaxPriority**
 - **getName, getParent**





Deadlocks

- A 'deadlock' occurs when two threads have a circular dependency on a pair of synchronized objects
- It is difficult to debug a deadlock for the following reasons:
 - It occurs very rarely, when the two threads time-slice in just the right way
 - It may involve more than two threads and two synchronized objects
- If a multithreaded program locks up frequently, immediately deadlock condition is to be checked

